

A `nofib` Benchmark Suite for PureLang

Design, Implementation, and Verification Notes

PureCake `examples/benchmark/nofib`

June 20, 2026

Abstract

This document records the construction of a PureLang port of part of the GHC `nofib` benchmark suite, living in `examples/benchmark/nofib/`. It explains the motivation, the language and tooling constraints that shaped every design decision, the methodology used to write and *verify* each benchmark, the meaning of every file in the suite, and the precise deviations from the original Haskell programs. The aim is that a future reader can understand not only *what* is in the directory but *why* it takes the form it does, and can reproduce or extend it with confidence.

Contents

1	Motivation	2
2	Background: PureLang and the compilation pipeline	2
3	Language constraints and their consequences	2
4	Methodology	3
4.1	Toolchain bring-up and the version-skew problem	3
4.2	Static validity signal	3
4.3	Dynamic verification and reference outputs	4
5	Suite structure and file formats	4
6	The benchmarks	5
6.1	Direct ports	5
6.2	<code>exp3.8</code> : Peano arithmetic	5
6.3	<code>gen_regexps</code> : strings as code points	5
6.4	<code>digits-of-e1</code> : continued-fraction spigot	6
6.5	<code>digits-of-e2</code> : strict <code>case</code> vs. lazy patterns	6
6.6	<code>wheel-sieve1</code> : untying the <code>primes</code> knot	6
6.7	<code>paraffins</code> : list memo for an array knot	6
6.8	<code>bernouilli</code> : explicit rationals	6
6.9	<code>primetest</code> : big-integer modular exponentiation	6
6.10	<code>sorting</code> : eight sorts without file I/O	7
7	The driver: <code>run.py</code>	7
8	Workload calibration	7
9	What was not ported, and why	8

10	Reproduction	8
11	Limitations and future work	8

1 Motivation

The `nofib` suite [1] is the long-standing collection of real Haskell programs used to measure lazy functional language implementations. Its purpose, in Partain’s words, is to give implementors “a common set of real . . . programs to attack and study” and to enable *reproducible* comparison. A `nofib` entry is not just a program: it is a program *plus* a workload *plus* an expected output *plus* rules for running and reporting.

PureCake is a verified compiler from PureLang—a small, lazy, Haskell-like language—to CakeML. Prior to this work, `examples/` contained a handful of demonstration programs and `examples/benchmark/` contained `benchmark.py`, which measures the effect of individual PureCake *optimisation flags* on a few programs. What was missing was a broad, output-checked corpus in the style of `nofib`: a suite that exercises the whole compiler on a variety of real programs and that doubles as an end-to-end correctness test (does the compiled program still compute the right answer?) and a performance test (how fast, and how much heap?).

The goal of this work was therefore to port as much of `nofib` as PureLang can faithfully express, with recorded workloads and reference outputs, plus a driver to compile, run, and check the suite.

2 Background: PureLang and the compilation pipeline

A PureLang program is compiled by the `pure` front end into a CakeML abstract syntax tree (serialised as an s-expression), which the `cake` compiler turns into machine code. The relevant top-level definition is `compile` in `compiler/pure_compilerScript.sml`; its phases are: parse, transform, inline, *type inference* (compilation aborts if this fails), dead-code elimination, demand (strictness) analysis, and lowering to CakeML. Two consequences matter here:

1. The `pure` front end performs full type inference, so a program that type-checks through `pure` is syntactically and statically valid.
2. The runnable artifact is produced by a two-stage external toolchain (`pure` then `cake`), whose serialised-AST interface evolves over time—a fact that materially affected our methodology (§4.1).

3 Language constraints and their consequences

The shape of every benchmark is dictated by what PureLang can and cannot express. The authoritative reference is `examples/syntax.hs`; the binding constraints are as follows.

Only one numeric type. PureLang’s sole numeric literal type is the arbitrary-precision mathematical `Integer` (confirmed in `language/pure_configScript.sml`, whose literal atoms are `Int` and `Str` only). There is no `Double`, `Float`, or `Rational`. *Consequence:* `nofib` programs that are intrinsically floating point cannot be ported; programs using `Data.Ratio` must represent rationals explicitly as reduced (*numerator, denominator*) pairs.

No module system and no type classes. There is no `import` mechanism, and the type-class extension to PureLang is work-in-progress and not available in the compiler used here. *Consequence:* each benchmark is a single, self-contained `.hs` file that inlines every helper it

uses (`map`, `foldr`, `toString`, `++`, and so on). There is no shared prelude, and—because there is no overloading—each file defines its own monomorphic `toString`, `++`, etc.

Strings are byte arrays. A PureLang `String` is a packed byte string (like Haskell’s `Text`), not a list of characters. Character-level work is done on `Integer` code points via built-ins such as `##(__Elem)`, `##(__Len)`, `##(__Concat)`, `##(__Implode)`, and `##(__Substring)`. *Consequence:* string-processing benchmarks operate on `[Integer]` of code points and convert at the boundary.

Input is a command-line argument, not standard input. Haskell `nofib` programs read a workload from `stdin` (a `.stdin` file). PureLang reads `argv` entries through the foreign call `Act (##(cline_arg) s)`, where the argument index equals the length of the string `s`; thus `Act (##(cline_arg) " ")` returns `argv[1]`. *Consequence:* every benchmark takes *one* command-line argument that scales the work—an integer for most programs, a regular-expression string for `gen_regexps`.

Strict, shallow pattern matching. `case` is strict, scrutinee patterns are shallow (a constructor applied to variables; no nested constructor patterns), and matches must be exhaustive or end in a wildcard. There are no list comprehensions and no guard syntax. *Consequence:* comprehensions are desugared by hand to `concatMap/filter/if`; guards become nested `if`; nested patterns (e.g. `Wheel s ns : ws`) are split into separate `cases`. Crucially, Haskell’s lazy *irrefutable* patterns (`let (x:xs) = e`) have no direct equivalent, because `case` forces its scrutinee; programs that rely on them must be restructured (§6).

4 Methodology

4.1 Toolchain bring-up and the version-skew problem

Running a PureLang program requires a `pure` binary (front end) and a `cake` binary (CakeML back end) that agree on the serialised-AST format. The `examples/Makefile` obtains these via `make download`, which fetches the *latest* `pure.S` and the *latest* CakeML release. At the time of writing these are not contemporaneous: the published `pure.S` dates from September 2024, whereas the latest `cake` release dates from June 2026, and the s-expression AST changed in between. Linking the latest `cake` additionally failed against the repository’s `basis_ffi.c` because of a newer foreign symbol (`fficustom`).

To obtain a *working, self-consistent* toolchain for development and reference-output generation we:

1. used the published `pure.S` (Sept. 2024) for the front end;
2. downloaded a contemporaneous CakeML release (v2648, Oct. 2024) for the back end, which both accepts that front end’s output and links cleanly against the existing FFI;
3. (only relevant to the *latest* `cake`) noted that a one-line `fficustom` stub resolves the link error.

This is an environmental work-around, not a change to the repository. The suite’s driver (§7) deliberately compiles through the standard `examples/Makefile`; the README records the caveat and recommends building `pure.S` locally from `compiler/binary` so that it matches a same-vintage CakeML checkout.

4.2 Static validity signal

The `pure` front end exits with status 0 even on an ill-typed or out-of-scope program; on failure it simply emits an almost-empty s-expression (observed as 5 bytes, versus thousands for a valid

program). We used *output size* from **pure** as the static-validity signal during development: a tiny output means a parse, scope, or type error. This caught the majority of mistakes (e.g. a use of **not** before it was defined) before ever invoking the back end.

4.3 Dynamic verification and reference outputs

Every benchmark was compiled *and executed* end to end, and its result checked against an independent expectation (known mathematical values or OEIS sequences). Examples of the checks performed:

- **queens**: $n = 8$ yields 92 solutions.
- **wheel-sieve1/primes**: the 0-indexed n th prime (e.g. **prime 100** = 547).
- **paraffins**: radical counts 1, 1, 1, 2, 4, 8, 17, 39, 89, ... and alkane counts 1, 1, 1, 2, 3, 5, 9, 18, ... (OEIS A000602).
- **bernouilli**: $B_{12} = -691/2730$, etc.
- **primetest**: correct primality of Mersenne numbers (M_{13} prime, $M_{11} = 23 \cdot 89$ composite, ...).
- **digits-of-e1/e2**: digits of e match 2.718281828459045....

The reference output files were then generated by running the verified executables at each workload size, not written by hand.

5 Suite structure and file formats

The suite mirrors **nofib**'s division into **imaginary** (small, self-contained programs) and **spectral** (algorithmic kernels). The **real** subset is not attempted (§9). The layout is:

```
benchmark/nofib/
  run.py                # compile + run + check driver
  README.md
  imaginary/<name>/
    <name>.hs           # the benchmark, one self-contained file
    sizes.cfg           # fast / norm / slow command-line arguments
    <name>.faststdout   # expected output for the fast workload
    <name>.stdout       # expected output for the norm workload
    <name>.slowstdout   # expected output for the slow workload
  spectral/<name>/ ...  # same per-benchmark layout
```

<name>.hs. A complete PureLang program with a **main :: IO ()** that reads one argument, performs the computation, and prints a short result line. The lower portion of each file is a block of standard I/O helpers (**fromString**, **toString**, **implode**, **reverse**, **read_arg1**, **print**, **++**, **\$**) copied verbatim across benchmarks, since there is no shared prelude.

sizes.cfg. A headerless key/value file giving the single command-line argument for each run mode, for example:

```
# Single command-line argument supplied to tak for each run mode.
fast = 8
norm = 9
slow = 10
```

The three modes follow **nofib**'s intent—**fast** for a quick smoke test, **norm** as the default (a second or two), **slow** for a heavier run.

`<name>.faststdout`, `<name>.stdout`, `<name>.slowstdout`. The expected program output for the `fast`, `norm`, and `slow` workloads respectively. The naming follows `nofib` convention (where `.stdout` is the norm-mode reference). `run.py` selects the file matching the chosen mode and compares it byte-for-byte against the program’s standard output.

6 The benchmarks

Table 1 lists the thirteen ported programs. The argument column gives the meaning of the single command-line input. The remaining subsections record, for the non-trivial cases, exactly how the port differs from the Haskell original and why.

Subset	Name	Arg	Computes
imaginary	<code>tak</code>	n	$\text{tak}(3n)(2n)n$ (Takeuchi)
imaginary	<code>rfib</code>	n	$\text{nfib } n$ (integer)
imaginary	<code>exp3_8</code>	n	3^n via Peano numerals
imaginary	<code>primes</code>	n	n th prime by iterated filtering
imaginary	<code>queens</code>	n	number of n -queens solutions
imaginary	<code>gen_regexps</code>	rx	size of a generalised-regexp expansion
imaginary	<code>digits-of-e1</code>	n	n digits of e (continued fraction)
imaginary	<code>digits-of-e2</code>	n	n digits of e (factorial base)
imaginary	<code>wheel-sieve1</code>	n	n th prime (lazy wheel sieve)
imaginary	<code>paraffins</code>	n	radical / paraffin counts up to n
imaginary	<code>bernouilli</code>	n	n th Bernoulli number
spectral	<code>primetest</code>	p	is $2^p - 1$ (Mersenne) prime?
spectral	<code>sorting</code>	n	run 8 sorts on n integers, check agreement

Table 1: The thirteen ported benchmarks.

6.1 Direct ports

`tak`, `rfib`, `primes`, and `queens` are close translations. `tak` computes $\text{tak}(3n)(2n)n$ so that $n = 8$ gives the canonical `tak 24 16 8`. `rfib` computes the integer `nfib` (the original returns a `Double`; with only `Integer` available we compute $\text{nfib } n = \text{nfib}(n - 1) + \text{nfib}(n - 2) + 1$, which counts calls). `primes` faithfully reproduces the deliberately slow `map head (iterate the_filter [2..n^2]) !! n`. `queens` reproduces the LML formulation $\text{nsoln } nq = \text{length } (\text{gen } nq)$ with the list comprehension in `gen` desugared to nested `concatMap`.

6.2 exp3_8: Peano arithmetic

A faithful port: naturals are `data Nat = Z | S Nat`, with addition and multiplication on `Nat`, and 3^n computed by repeated multiplication. The result is converted back to an `Integer` for printing ($3^8 = 6561$). Because the representation is unary, the `slow` workload ($3^{16} \approx 4.3 \times 10^7$ constructors) requires a multi-gigabyte heap; this is by design, exercising allocation.

6.3 gen_regexps: strings as code points

The expansion of a generalised regular expression is computed over `[Integer]` (lists of character codes) rather than `String`, because `PureLang` strings are not lists of characters. Following the RJE variant noted in the original source, the program prints the *number of characters* produced rather than the (potentially enormous) expansions themselves; this is deterministic and avoids huge output.

6.4 digits-of-e1: continued-fraction spigot

A close port of the Thurston algorithm: the linear-fractional transformation `ratTrans` operates on a four-tuple of `Integer` and the continued fraction of e . The only change is presentational—the digits are printed as 2.71828... instead of being hashed.

6.5 digits-of-e2: strict case vs. lazy patterns

The Hughes factorial-base algorithm relies on a lazy irrefutable pattern (`nextcarry:fraction = carryPropagate (base+1) ds`) that is *never forced past the end* of the otherwise-infinite stream. PureLang’s `case` is strict, so we give `carryPropagate` an explicit base case for the empty list. Correctness is preserved because the algorithm already consumes a finite $2n$ -element prefix as “an upper bound on what the pipeline might consume”: processing that finite prefix strictly, with a terminating base case, yields the first n digits correctly (the artificial boundary stays in the unused tail).

6.6 wheel-sieve1: untying the primes knot

This program defines `primes` self-referentially (`primes = sieve (wheels primes) primes (squares primes) n`); it works in Haskell only because the first element of each wheel’s multiple-list, `s:[s*2,s*3..bound]`, is available *without* forcing `bound` (which depends on `head primes`). Our first, naive desugaring used `enumFromTo 1 k0`, which forces `k0` immediately and produces a black hole (the program looped/aborted for all n). The fix mirrors the original’s laziness exactly: the index list is written `1 : enumFromTo 2 k0` so that the first multiple is produced without forcing `k0`, untying the knot. The nested pattern `Wheel s ns : ws` is split into two cases.

6.7 paraffins: list memo for an array knot

The original memoises radical lists in a `Data.Array` defined in terms of itself. PureLang has no immutable arrays, so the memo is a lazy self-referential *list* indexed by a recursive helper; this is well-founded because `rads_of_size_n` only ever indexes strictly smaller sizes. A key observation makes the port tractable: only the *lengths* of the radical lists affect the reported counts, never the radical values, so each radical is represented by a dummy 0. Outputs match OEIS A000602.

6.8 bernouilli: explicit rationals

`Data.Ratio` is replaced by `data Rat = Rat Integer Integer` with reduced fractions (positive denominator, divided through by `gcd`). The self-referential infinite tables of the original (`powers`, `neg_powers`, `pascal`) are reproduced directly, relying on PureLang’s laziness; only finite prefixes are consumed. Output matches the `Ratio show` format, e.g. 5 % 66, and reproduces the known Bernoulli numbers.

6.9 primetest: big-integer modular exponentiation

The original runs a Rabin–Miller test on Mersenne numbers read from a file. We keep the bignum essence—square-and-multiply modular exponentiation—and test whether $M_p = 2^p - 1$ is prime using Miller–Rabin with a fixed set of small witnesses (deterministic and reproducible, avoiding a random-number generator). This is a probabilistic test in general but classifies all the Mersenne exponents in the workloads correctly.

6.10 sorting: eight sorts without file I/O

All eight sorts from the original (`quickSort`, `quickSort2`, `quickerSort`, `insertSort`, `treeSort`, `treeSort2`, `heapSort`, `mergeSort`) are ported over `[Integer]`. Lacking `readFile`, the program generates a deterministic scrambled list of n integers with a linear-congruential generator, runs all eight sorts, checks they agree, and prints a checksum. This exercises every sort on identical input and self-validates.

7 The driver: `run.py`

`run.py` discovers every benchmark (a directory containing `<name>.hs`), and for the chosen mode:

1. reads the workload argument from `sizes.cfg`;
2. compiles the benchmark by invoking the existing `examples/Makefile` target (so it uses whatever `pure/cake` the user has, and forwards `PUREOPT` optimisation flags);
3. runs the resulting executable from `examples/out/...` with the argument and a configurable CakeML heap size;
4. compares standard output byte-for-byte with the mode's reference file and reports `PASS/FAIL`.

Useful invocations:

```
./run.py           # compile, run, check everything at norm
./run.py --mode fast # quick smoke test
./run.py -k tak -k primes # only matching benchmarks
./run.py --time --iterations 5 # best-of-5 wall-clock timing
./run.py --no-compile # reuse already-built executables
./run.py --list       # list benchmarks
```

The default heap is 4096 MB because the slow workloads of `exp3_8` and `gen_regexps` need several gigabytes; the executable itself is independent of the mode, so `--no-compile` can be combined with any `--mode`.

8 Workload calibration

Sizes were chosen empirically so that, on the development machine, `norm` runs in roughly one to three seconds (consistent with `nofib`'s “norm” intent), `fast` in a fraction of a second, and `slow` in a handful of seconds. Table 2 lists the chosen arguments and the measured `norm` wall-clock time. Note the steep growth of some programs (`tak`, `paraffins`, `queens`), which constrains how far the `slow` sizes can be pushed.

Name	fast	norm	slow	norm time (s)
<code>tak</code>	8	9	10	2.13
<code>rfib</code>	28	34	37	1.31
<code>exp3_8</code>	11	14	16	0.66
<code>primes</code>	300	3000	5000	1.03
<code>queens</code>	9	11	12	2.80
<code>gen_regexps</code>	<code>[a-z][a-z][0-9]</code>	<code>[a-z][a-z][a-z][a-z]</code>	<code>[a-z][a-z][a-z][a-z][0-3]</code>	0.68
<code>digits-of-e1</code>	100	800	1500	1.11
<code>digits-of-e2</code>	100	900	1600	0.83
<code>wheel-sieve1</code>	2000	60000	150000	1.63
<code>paraffins</code>	14	18	19	0.39
<code>bernouilli</code>	100	500	800	0.69
<code>primetest</code>	521	1279	2281	0.48
<code>sorting</code>	500	4000	10000	1.80

Table 2: Workload arguments per mode and measured norm-mode time.

9 What was not ported, and why

Floating-point imaginary programs

`integrate`, `kahan`, and `x2n1` are intrinsically floating point and have no faithful integer rendering; PureLang has no floating-point type.

`wheel-sieve2`

The “Mark II” sieve hinges on a lazy, infinite `roll/dropWhile` spiral whose subtle demand behaviour does not survive translation to strict `case`. Since `wheel-sieve1` already represents the lazy-wheel-sieve family, the marginal value did not justify the risk, and it is omitted.

The real subset

These are application-scale programs requiring a module system, file/standard input, and frequently floating point— none of which PureLang currently offers.

10 Reproduction

1. Obtain a matching `pure/cake` pair. The robust route is to build `pure.S` locally from `compiler/binary` against a CakeML checkout, then build the `cake` binary from the same checkout. Otherwise `make download` fetches prebuilt artifacts, subject to the version-skew caveat of §4.1.
2. From `examples/benchmark/nofib`, run `./run.py --mode fast` for a smoke test, then `./run.py` for the full norm-mode check.
3. To regenerate a reference after changing a benchmark, rebuild its executable via the Makefile and redirect its output at the appropriate size into the corresponding `*stdout` file (see the suite README).

11 Limitations and future work

The reported strings are close to, but not byte-identical with, GHC’s `nofib` outputs: results are printed directly rather than hashed, and some programs print a derived summary (a character count, a checksum) rather than the full data. This is intentional and adequate for regression testing within PureCake, but means the suite is not a cross-implementation oracle. Should PureLang gain floating point, a module system, or type classes, the float-only `imaginary` programs, `wheel-sieve2`, and eventually parts of the `spectral` and `real` subsets become candidates for inclusion. The suite is structured so that adding a benchmark is mechanical: create a self-contained `.hs`, add a `sizes.cfg`, record three reference outputs, and confirm with `run.py`.

References

- [1] W. Partain. *The `nofib` Benchmark Suite of Haskell Programs*. In *Functional Programming, Glasgow 1992*, British Computer Society, 1993. (Included in this repository as `nofib.pdf`.)